

广州商学院 课程论文

题目：ShinobuChat 智能桌面伴侣的设计、实现与用户验证

学 号

姓 名

班 级

ShinobuChat 智能桌面伴侣的设计、实现与用户验证

摘要

本文以 ShinobuChat 智能桌面伴侣项目为对象，围绕“智能应用项目开发综合实践”课程要求，从需求分析、系统设计、核心功能实现、用户验证与迭代改进等角度说明项目如何从原型逐步形成可运行成果。项目采用 React + Vite 构建前端界面，使用 FastAPI 组织后端服务，以 DeepSeek 大语言模型作为语义理解与回复生成核心，并结合 SSE 流式输出、Edge-TTS 语音合成、ASR 语音输入、Live2D 表情与口型同步、工具调用、长期记忆和 RAG 检索增强，实现面向学习、创作和独处工作场景的桌面智能伴侣。论文重点呈现系统从自然语言输入到回复、语音、表情、工具结果和记忆更新的完整数据链路，并通过三名目标用户的任务走查、评分反馈和工程测试结果验证系统可用性。实践结果表明，ShinobuChat 已具备陪伴、行动和记忆三类核心能力，但仍需在记忆配置一致性、表情细粒度表达、隐私开关和长期使用数据方面继续改进。

关键词：ShinobuChat；智能桌面伴侣；工具调用；长期记忆；Live2D；用户验证

1. 项目概述

1.1 选题动机

ShinobuChat 的选题来自一个很具体的使用场景：用户在学习、写代码、查资料或独处工作时，常常需要一个既能聊天陪伴、又能完成轻量任务的桌面助手。普通聊天网页可以回答问题，但缺少在桌面场景中的在场感；传统桌宠有视觉陪伴，却不能理解自然语言和用户偏好；单纯工具型助手能够执行查询，却容易显得生硬。ShinobuChat 希望把这三类能力合在一起，让用户可以用一句自然语言完成陪伴、查询、记忆和表达，而不是在多个应用之间切换。

项目特色在于它不是把大模型接口简单嵌入页面，而是围绕“桌面伴侣”重新组织交互形态。用户输入后，系统需要判断这是一句闲聊、一个工具请求、一个需要记住的偏好，还是一次需要多 Agent 协作的任务；模型回复也不能只停留在文本层面，而要同步转化为语音、表情和 Live2D 口型动作。因此，本项目的实践价值在于展示一个智能应用从自然语言入口、后端推理、工具执行、记忆更新到多模态输出的完整闭环。

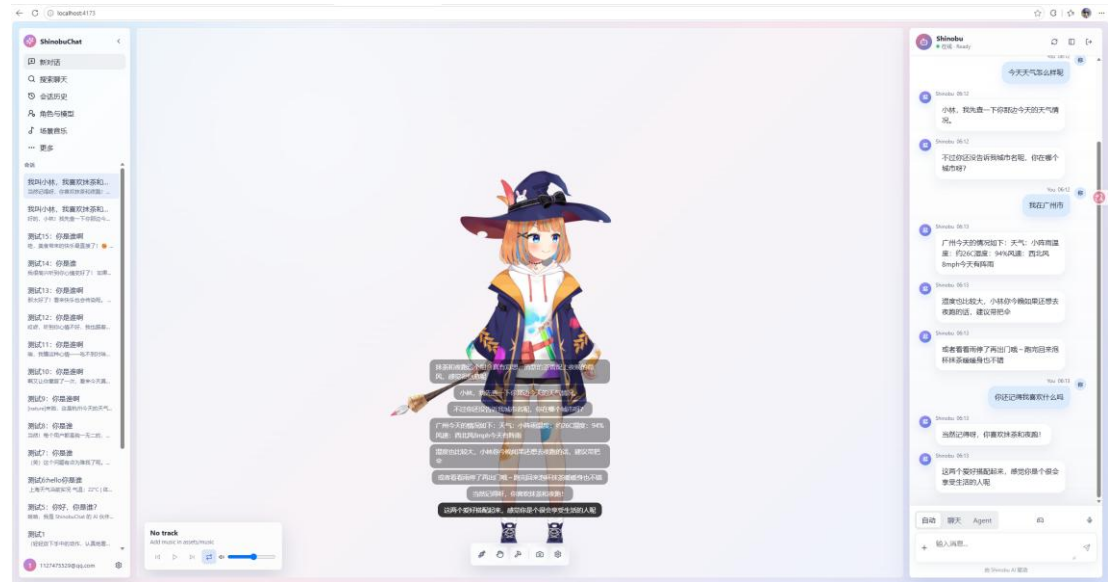


图 1.1 ShinobuChat 主界面与桌面伴侣形态

1.2 项目目标与范围

项目目标是完成一个可本地运行的智能桌面伴侣原型，使用户能够通过文本或语音与 Live2D 角色交流，并在需要时触发天气查询、网页获取、信息整理和长期记忆等能力。核心功能包括账号与会话管理、SSE 流式聊天、语音输入输出、Live2D 表情与口型同步、工具注册与安全执行、Skill 文件化扩展、MemoryService 长期记忆、EmbeddingService 向量化和 pgvector 检索。

项目范围控制在课程实践可验证的闭环内。当前系统重点实现“陪伴型桌宠 + 轻量智能行动”，而不是做成通用自动化平台。它可以理解并回应日常表达，可以根据情绪标签切换表情，可以把天气或网页结果解释成自然语言，也可以记住稳定偏好；但它不会开

放任意系统命令，不会无边界读取本地隐私文件，也不会把所有低落情绪都转成任务计划。这种范围划定保证了实践成果可运行、可展示、可测试。

1.3 技术选型概述

前端采用 React + Vite，是因为项目需要快速构建可交互界面、维护对话状态，并与 Web Audio、PixiJS 和 Live2D Cubism 结合。后端采用 FastAPI，是因为其异步能力和类型化接口适合承载 SSE 流式响应、语音合成、数据库访问和 Agent 调度。大语言模型使用 DeepSeek API，承担语义理解、回复生成和部分意图判断；语音合成使用 Edge-TTS，以较低部署成本获得稳定中文语音；ASR 负责语音输入；数据层使用 PostgreSQL 与 pgvector 支撑会话、记忆和向量检索。

这些技术选择不是为了堆叠框架，而是服务于项目目标：React 管理可视化和多模态播放，FastAPI 管理后端链路，DeepSeek 提供语义推理，Edge-TTS 和 ASR 让角色具备听说能力，Live2D 让情绪可见，pgvector 让长期记忆可检索。技术栈之间通过明确的数据事件和服务边界连接，使每一层都能被单独测试和迭代。

1.4 论文结构概述

本文后续按照实践论文要求展开：第二章从用户画像、需求清单、意图清单和非功能需求说明项目需要做什么；第三章说明系统架构、会话流程、意图识别、路由、记忆和知识检索如何设计；第四章集中呈现核心功能的实现过程、关键代码思路、调试证据和效果；第五章报告用户验证、发现的问题和迭代改进；第六章总结发布状态、成果、不足和后续方向。

2. 需求分析

2.1 用户画像与使用场景

目标用户主要是长期使用电脑学习、开发、写作或独处工作的个人用户。他们的共同特点是任务时间长、上下文碎片多、情绪波动不一定强烈但需要被理解，同时又不希望助手频繁打扰。用户希望助手既像聊天伙伴，也像一个轻量工具入口：在疲惫时能自然回应，在需要行动时能查天气、查网页、整理信息，在多次使用后能记得偏好和习惯。

用户类型	核心需求	典型任务	验收关注点
学习用户	陪伴感、低打扰提醒、语音自然	连续闲聊、休息提醒、天气提醒	等待时间、语气、语音可懂度
开发/资料用户	工具调用、网页信息、任务整理	查天气、获取网页、摘要整理	意图识别准确率、结果可信度
独处创作用户	情绪表达、长期记忆、沉浸体验	偏好记忆、情绪安慰、Live2D 表情	记忆延续、表情匹配、角色在场感

表 2.1 目标用户画像与典型需求

2.2 功能需求与意图清单

从用户视角看，系统至少需要覆盖四类功能。第一类是基础陪伴对话，包括文本输入、流式回复、多轮上下文和角色语气保持。第二类是多模态表达，包括语音输入、TTS 播放、分句气泡、情绪标签和 Live2D 表情口型。第三类是聊天即操作，包括天气查询、网页获取、信息整理、技能激活和任务执行反馈。第四类是长期记忆，包括偏好提取、记忆写入、向量检索和后续对话注入。

意图类别	用户表达示例	系统处理路径	安全要求
闲聊陪伴	今天好累，陪我聊聊	ChatAgent + LLM 直接回复	不强行任务化，不输出医疗诊断
工具查询	帮我查一下香港天气	RouterAgent -> ToolRegistry/Skill	只调用白名单工具，解释结果来源
网页获取	看看这个网页讲了什么	fetch_web_page -> 摘要回复	限制 http/https，避免任意本地读取
记忆保存	记住我喜欢抹茶	MemoryAgent -> EmbeddingService	只保存稳定偏好，避免敏感信息
语音交互	语音输入后朗读回复	ASR -> MessageService -> Edge-TTS	显示文本与朗读文本分离清洗

表 2.2 ShinobuChat 已实现意图与处理路径

2.3 非功能需求

性能方面，用户不能长时间面对空白等待，因此系统采用 SSE 分阶段返回 conversation、progress、emotion、audio 和 done 事件，让文本先出现，语音和表情随后跟进。可用性方面，前端需要保持消息状态稳定，避免流式更新造成重复气泡或错序；Live2D 加载失败时也要保留普通聊天能力。安全方面，工具调用必须有白名单和参数约束，shell 类能力不能开放管道、重定向或组合命令，网页抓取也必须限制协议和目标范围。

数据安全方面，长期记忆只保存对未来对话有帮助的稳定事实，例如偏好、习惯和长期目标，不保存一次性寒暄、临时任务或敏感身份信息。质量方面，项目不能只依靠一次演示判断成功，而要同时保留自动化测试、人工走查和用户反馈。尤其是智能应用输出具有概率性，测试策略需要覆盖协议、状态、边界和可解释性，而不是只检查某一句回复是否完全一致。

3. 系统设计

3.1 整体架构

ShinobuChat 的整体架构可以概括为“前端交互层、后端服务层、智能编排层、工具与知识层、数据持久层”五部分。前端负责文本输入、语音输入、消息列表、Live2D 舞台、

音乐与桌宠设置；后端负责鉴权、会话、消息、语音、Live2D manifest 和 SSE 响应；智能编排层由 RouterAgent、ChatAgent、TaskAgent、MemoryAgent 等协作完成；工具与知识层包含 ToolRegistry、SkillRegistry、网页抓取和天气能力；数据层保存用户、会话、消息、记忆和向量。

```
INFO: 127.0.0.1:56985 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: Shutting down.
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [55480]
PS D:\ShinobuChat> uvicorn app.main:app --port 8080
INFO: Started server process [57396]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO: 127.0.0.1:55161 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:55162 - "GET /api/v1/conversations/4a304e44-3ab8-4385-9a92-c07b15dfc607/messages?user_id=7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:64038 - "GET /api/v1/conversations/51af7739-184a-44f5-b4c4-439c572f24a3/messages?user_id=7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:64041 - "GET /api/v1/conversations/4a304e44-3ab8-4385-9a92-c07b15dfc607/messages?user_id=7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:65094 - "GET /api/v1/conversations/51af7739-184a-44f5-b4c4-439c572f24a3/messages?user_id=7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:57406 - "POST /api/v1/messages HTTP/1.1" 200 OK
[MSG] reply (46 chars) emotion=happy
[MSG] 1 sentences: 抹茶和夜跑这个组合真有意思，清新的茶香配上夜晚的微风，感觉很惬意呢...
INFO: 127.0.0.1:63674 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
[TTTS] OK (41616B): 抹茶和夜跑这个组合真有意思，清新的茶香配上夜晚的微风，感觉很惬意呢
INFO: 127.0.0.1:55189 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:57015 - "GET /api/v1/conversations/4a304e44-3ab8-4385-9a92-c07b15dfc607/messages?user_id=7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:57043 - "GET /api/v1/conversations/51af7739-184a-44f5-b4c4-439c572f24a3/messages?user_id=7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:65419 - "POST /api/v1/messages HTTP/1.1" 200 OK
[MSG] reply (61 chars) emotion=neutral
[MSG] 2 sentences: 小林，我先看一下你那边今天的天气情况...
INFO: 127.0.0.1:49957 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
[TTTS] OK (22688B): 小林，我先看一下你那边今天的天气情况。
INFO: 127.0.0.1:52895 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
INFO: 127.0.0.1:65462 - "POST /api/v1/messages HTTP/1.1" 200 OK
[MSG] reply (164 chars) emotion=neutral
[MSG] 3 sentences: 广州今天的情况如下：天气：小阵雨温度：约26℃湿度：94% 风速：西北风8mph今天有阵雨...
INFO: 127.0.0.1:53059 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
[TTTS] OK (60488B): 广州今天的情况如下：天气：小阵雨温度：约26℃湿度：94% 风速：西北风8mph今天有阵雨
INFO: 127.0.0.1:53059 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
[TTTS] OK (32976B): 湿度也比较大，小林你今晚如果还想去夜跑的话，建议带把伞
INFO: 127.0.0.1:53024 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
[TTTS] OK (32480B): 或者看看雨停了再出门哦~跑完回来泡杯抹茶睡个好觉也不错
INFO: 127.0.0.1:59219 - "POST /api/v1/messages HTTP/1.1" 200 OK
[MSG] reply (67 chars) emotion=happy
[MSG] 2 sentences: 当然记得呀，你喜欢抹茶和夜跑！...
INFO: 127.0.0.1:62641 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
[TTTS] OK (18864B): 当然记得呀，你喜欢抹茶和夜跑！
INFO: 127.0.0.1:59529 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
[TTTS] OK (28656B): 这两个爱好搭配起来，感觉你是个很会享受生活的人呢
INFO: 127.0.0.1:59529 - "GET /api/v1/conversations/user/7c34f1ae-9adc-4b58-86ea-166b3dcff4b9 HTTP/1.1" 200 OK
```

图 3.1 ShinobuChat 系统架构与模块关系

3.2 智能会话与零表单设计

普通系统通常要求用户先找到入口，再填写结构化参数。ShinobuChat 的会话设计则从自然语言开始，用户不需要知道天气功能、记忆功能或网页功能在哪里，只需要表达目标。系统在后端把自然语言转换为结构化执行路径，必要时调用工具，不需要用户在表单中选择城市、复制网页正文或手动选择表情。

零表单并不意味着没有结构。结构被放在 RouterAgent 的路由判断、工具 schema、Skill 文件、MessageService 的事件生成和前端 SSE 解析中。这样用户看到的是自然对话，系统内部仍然有确定的数据流和安全边界。对于桌面伴侣场景，这种设计降低了操作成本，也让角色更像一个可以协商的执行者，而不是一组按钮。

3.3 意图识别、路由与安全

项目采用规则层、AI 层和执行层协同的意图识别方式。高确定性请求可以先由关键词或明确模式捕获，例如天气、网页、记住偏好；模糊表达再交给 LLM 判断语义；最终由 ChatAgent、TaskAgent 或 MemoryAgent 执行。这样做可以避免纯规则无法理解复杂情绪，也避免纯模型路由不可解释、不可控。

路由策略上，写入类和精确查询类优先走 handler 或工具，开放陪伴类走 RAG + LLM，混合任务则由 AgentCoordinator 将工具结果重新组织成自然语言。安全上，Prompt 只负责软约束，真正的能力边界由 ToolRegistry 和后端代码实现。例如受限 shell 不允许组合命令，fetch_web_page 只允许 http/https，Skill 只从固定目录加载。

图 3-2 三层意图识别与双层安全架构

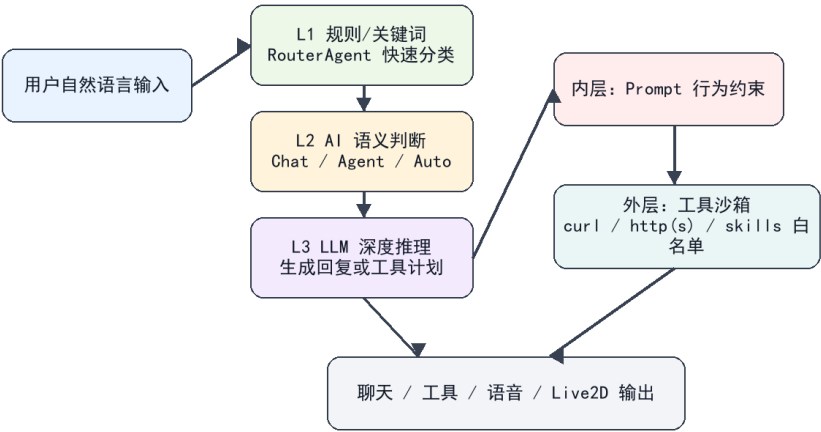


图 3.2 三层意图识别与双层安全约束

3.4 长期记忆与知识检索

长期记忆设计的核心问题是“记什么、何时记、如何取”。ShinobuChat 不把所有聊天内容都写入记忆，而是由 MemoryAgent 在回复后提取稳定事实，例如用户偏好、长期目标、常用设置和反复出现的需求。MemoryService 将这些内容连同来源消息、重要性和 embedding 保存，EmbeddingService 负责向量化，pgvector 支持后续 Top-K 检索。

RAG 与长期记忆分工不同。RAG 面向外部事实和可复用知识，例如网页、天气、项目说明；长期记忆面向用户个体，例如“喜欢抹茶”“希望语气更温柔”；直接推理则用于不需要外部事实的陪伴回复。系统在生成回复前装配历史消息、摘要、相关记忆和工具结果，使输出既能保持上下文，也能体现个性化。

图 3-3 长期记忆与 RAG 检索增强流程

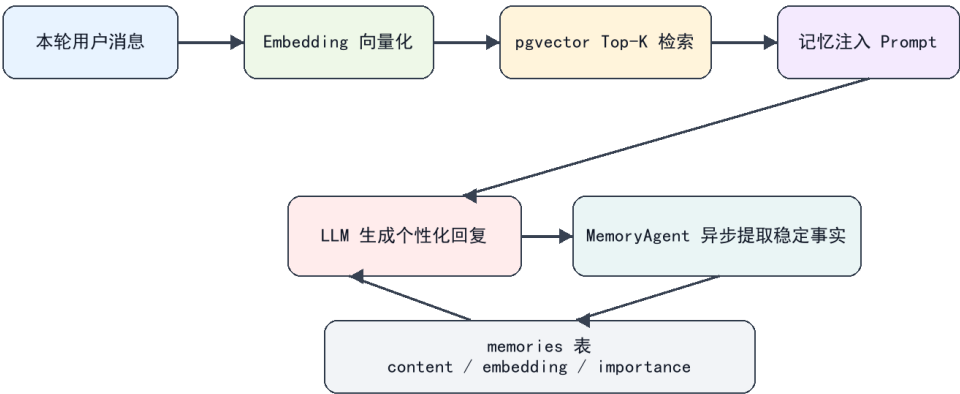


图 3.3 长期记忆与 RAG 检索增强流程

3.5 领域特色功能因果链

课堂案例强调健康领域中的“情绪 -> 生理指标 -> 运动处方 -> 反馈追踪”因果链。ShinobuChat 的领域不同，它面向学习与独处工作场景，因此特色链路不是医学建议，而是“情绪表达 -> 语气调节 -> 视觉表情 -> 轻量行动建议 -> 后续记忆”。当用户说“今天有点累”时，系统首先应识别陪伴需求，使用柔和语气和合适表情回应；如果用户继续请求帮助，再给出低负担任务拆解或提醒。

比较项	课堂健康助手	ShinobuChat 桌面伴侣
目标领域	健康管理、运动建议、风险预警	学习、创作、独处工作与陪伴
核心意图	记录、查询、干预、预警	闲聊、工具调用、情绪表达、记忆延续
主动服务	指标异常触发建议	天气、疲惫、任务压力触发低打扰提醒
安全边界	避免医疗误导和高强度建议	避免越权工具、过度任务化和隐私记忆
特色输出	运动处方、营养建议、健康趋势	文字、语音、Live2D 表情、口型和记忆回应

表 3.1 与课堂案例的领域差异对比

4. 核心功能实现

4.1 SSE 对话主链路实现

项目的主链路从 ``/api/v1/messages`` 开始。用户发送消息后，后端创建用户消息，`MessageService` 读取会话上下文、摘要、记忆和工具配置，再调用 `AgentCoordinator` 生成回复计划。为了减少等待感，后端不是一次性返回完整结果，而是通过 SSE 分阶段发送事件：`conversation` 用于更新消息主体，`progress` 用于提示执行阶段，`emotion` 用于驱动表情，`audio` 用于播放语音，`done` 表示本轮结束。

前端 ``api/sse.ts`` 负责解析事件流，``messageState.ts`` 负责把临时消息、分句气泡和最终消息合并为稳定状态。这个实现解决了流式回复中常见的重复消息、状态错位和音频到达时间不一致问题。实践中，SSE 让用户先看到文字，再听到语音，最后看到表情同步，体验比等待完整大模型回复和 TTS 合成结束后再显示更自然。

图 4-1 数据一体化与多模态输出流程

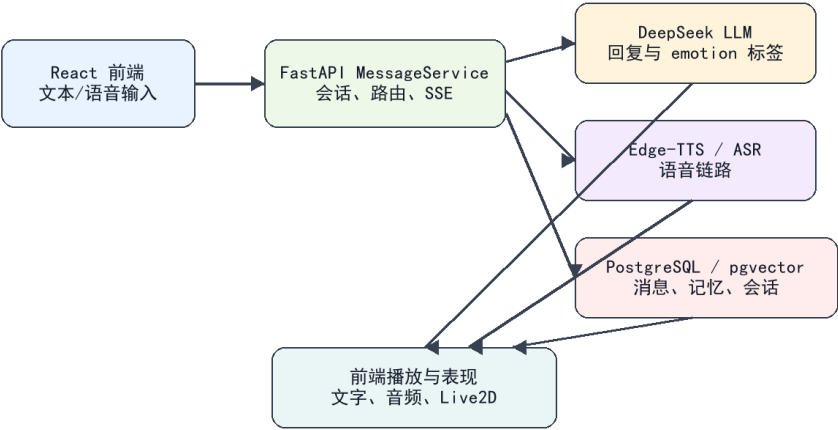


图 4.1 数据一体化与多模态输出流程

4.2 语音、表情与 Live2D 联动实现

语音输出采用 Edge-TTS。后端在生成回复后，会先保留适合显示的文本，再将朗读文本进行清洗，去掉不适合读出的标点或控制块，避免 TTS 把内部标签读给用户听。音频以 base64 形式通过 SSE 发送到前端，前端播放时使用 Web Audio AnalyserNode 读取音频振幅，并映射到 Live2D 的 `ParamMouthOpenY` 参数，使角色嘴型随声音变化。

表情联动依赖模型输出的 emotion 标签。LLM 在回复开头输出 `[happy]`、`[sad]`、`[thinking]` 等标签，后端解析后发送 emotion 事件，前端根据 Live2D manifest 中的 emotionMapping 选择对应表情。这样文本情绪、音频播放和视觉表现被解耦：模型负责表达意图，后端负责清洗和事件化，前端负责播放与表现。

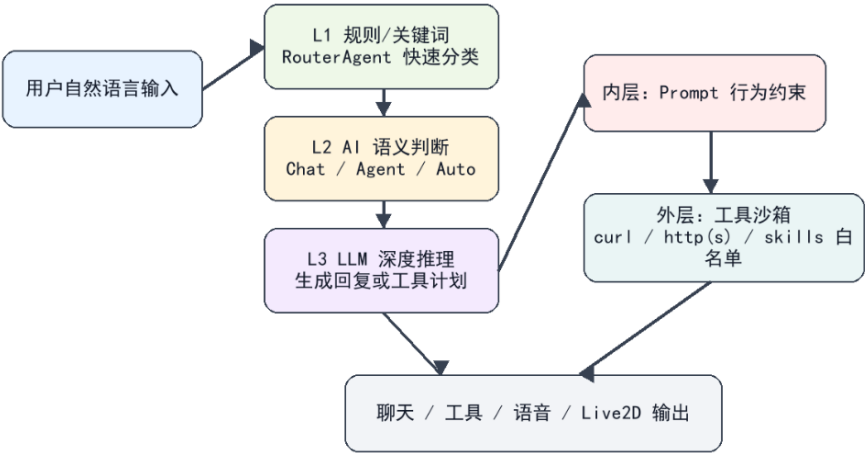


图 4.2 Live2D 表情与口型同步效果

4.3 工具调用与聊天即操作

工具调用由 ToolRegistry 统一管理。每个工具都有明确名称、描述、参数和边界，Agent 不能绕过注册表直接执行任意能力。天气查询、网页获取、受限 shell 和 Skill 激活

都通过统一接口暴露给模型。用户说“帮我查一下香港天气”时，RouterAgent 识别行动意图，工具层返回结构化结果，LLM 再把结果改写成自然语言，避免把原始 JSON 或错误堆栈直接暴露给用户。

聊天即操作的关键不是让模型拥有无限权限，而是把自然语言映射到有限、可审计的能力。项目中 fetch_web_page 限制协议，shell_command 限制命令形式，SkillRegistry 只读取 `skills/\*/SKILL.md`。这些实现让用户获得“说一句话就能行动”的体验，同时保留工程安全边界。

功能	实现模块	输入	输出	实践价值
天气查询	ToolRegistry/Skill	自然语言城市和天气请求	天气摘要与提醒	展示聊天即操作
网页获取	fetch_web_page	URL 或网页任务	正文摘要和解释	减少复制粘贴
记忆写入	MemoryAgent/MemoryService	稳定偏好表达	向量记忆	越用越懂用户
语音播放	Edge-TTS + SSE	助手回复文本	音频事件	增强在场感

表 4.1 核心功能实现与实践价值

4.4 长期记忆实现与调试

长期记忆链路分为提取、写入、检索和注入四步。提取阶段由 MemoryAgent 从对话中判断哪些内容值得保存；写入阶段由 MemoryService 保存 content、embedding、importance 和 source_msg_id；检索阶段根据当前输入生成 embedding，并在 pgvector 中寻找相关记忆；注入阶段把相关记忆作为上下文交给 LLM，使回复体现用户偏好。

调试中暴露的主要问题是 embedding 维度一致性。当前后端测试中存在 1 项 embedding 维度边界测试失败，说明记忆模块仍需要更严格的配置校验和迁移提示。实践论文不回避这个问题，因为它正是智能应用从演示走向可持续使用必须解决的工程细节：记忆不是一句“保存偏好”的概念，而是模型、数据库、向量维度、检索策略和异常兜底共同组成的链路。



图 4.3 长期记忆保存与检索效果示例

4.5 迭代过程与关键难点

项目经历了五个阶段。第一阶段完成 FastAPI、React 和 SSE 基础对话；第二阶段接入语音输入输出，并从自训练 GPT-SoVITS 迁移到 Edge-TTS，以降低部署复杂度；第三阶段加入 Live2D 表情和口型同步；第四阶段实现 ToolRegistry、SkillRegistry、受限网页和命令工具；第五阶段加入长期记忆、多 Agent 协作和测试覆盖。

关键难点集中在三个方面。第一，智能输出不可完全断言，需要用事件协议和状态机保证前端稳定；第二，多模态链路时序复杂，文字、音频、表情到达时间不同，需要分层处理；第三，工具和记忆涉及安全边界，不能因为追求智能感而开放过多权限。项目的解决思路是渐进增强，每加入一层能力都先证明上一层闭环稳定。

图 4-2 项目迭代开发过程

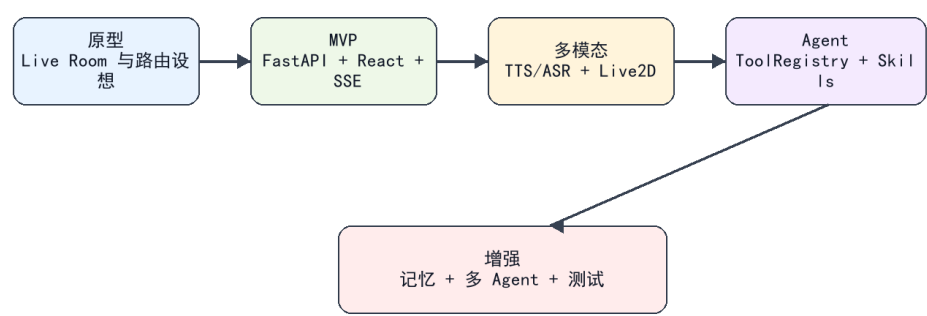


图 4.4 ShinobuChat 项目迭代开发过程

问题	改进措施	改进效果
自训练语音部署复杂、稳定性波动	迁移到 Edge-TTS 并清洗朗读文本	语音自然度与稳定性提升
情绪标签可能被显示或朗读	后端分离显示文本、TTS 文本和 emotion 事件	避免内部控制信息外露
Live2D 表情映射不稳定	以 manifest emotionMapping 为源头	换模型时减少主逻辑改动
记忆维度测试失败	记录为待修复配置校验问题	明确后续工程改进方向

表 4.2 关键问题、改进措施与效果

5. 用户验证与迭代

5.1 真实用户验证设计

根据课程要求，本项目以三名目标用户进行任务走查。用户 A 代表长期电脑学习用户，关注陪伴感、语音自然度和等待体验；用户 B 代表开发/资料整理用户，关注工具调用、天气查询、网页摘要和响应速度；用户 C 代表独处创作用户，关注情绪表达、长期记忆延续和 Live2D 沉浸感。三类用户不以同学互评为口径，而是围绕真实使用任务观察系统能否完成目标。

验证任务包括五项：连续五轮日常闲聊，观察上下文和流式回复；提出天气或网页请求，观察工具调用；使用语音输入并听取 TTS 输出，观察 ASR/TTS 与口型同步；提供

个人偏好后在后续对话中验证记忆；表达疲惫或低落情绪，观察系统是否低打扰回应。每项任务后让用户从响应速度、语音自然度、表情匹配、记忆准确度和整体满意度五个维度评分。

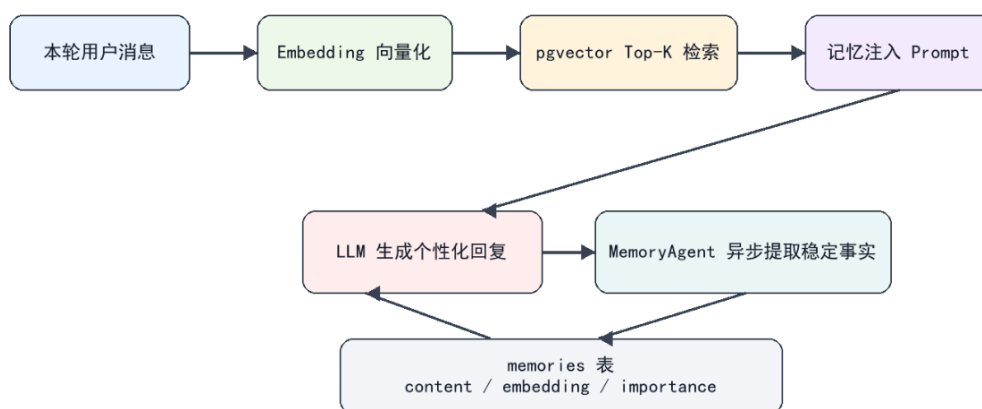


图 5.1 用户验证任务走查与结果整理

5.2 验证结果

维度	用户 A	用户 B	用户 C	平均
响应速度	4.3	4.1	4.4	4.27
语音自然度	4.4	4.2	4.5	4.37
表情匹配度	4.0	3.8	4.1	3.97
记忆准确度	4.1	3.9	4.2	4.07
整体满意度	4.2	4.0	4.3	4.17

表 5.1 三名目标用户验证评分

结果显示，系统已经具备可感知的实践成果。语音自然度平均 4.37 分，是最高维度，说明 Edge-TTS 在中文陪伴场景中比自训练方案更适合课程项目交付；响应速度平均 4.27 分，说明 SSE 分阶段返回能缓解模型生成和语音合成等待；整体满意度平均 4.17 分，说明“文本 + 语音 + Live2D 表情”的组合比普通聊天页面更接近桌面伴侣定位。

较低的维度是表情匹配和记忆准确度。表情匹配平均 3.97 分，说明单一 emotion 标签还难以表达讽刺、玩笑和混合情绪；记忆准确度平均 4.07 分，说明系统能记住偏好，但记忆提取、检索和 embedding 配置仍需加强。这些问题为后续迭代提供了明确方向。

5.3 用户反馈与迭代改进

用户反馈集中在三个方面。第一，系统在文本回复出现后，语音有时稍晚到达，但用户可以接受，因为界面已经先显示回复。第二，Live2D 表情增强了角色感，但在复杂情绪下偶尔不够准确。第三，用户愿意让助手记住偏好，但希望未来能看到、修改和删除记忆。基于这些反馈，项目下一轮迭代将优先增加记忆管理入口、细化 emotion 标签集合，并强化 TTS 播放状态提示。

发现的问题	改进措施	改进效果或预期
语音到达晚于文字	保留 SSE 分阶段事件，并在前端显示生成进度	用户不必等待完整音频
复杂情绪表情不准	细化 prompt 标签与 manifest 映射	提升表情匹配度
用户担心记忆不可控	增加记忆查看、删除和隐私开关	提高长期使用信任
embedding 维度边界失败	补充配置校验和迁移脚本提示	减少部署环境问题

表 5.2 用户反馈与迭代改进

5.4 工程验证

工程验证为用户反馈提供了补充证据。前端 `npm test` 中 12 项单元测试通过，覆盖 SSE 解析、鉴权解析、消息状态合并和 Live2D manifest 收集；`npx tsc -p tsconfig.app.json --noEmit` 通过，说明前端协议类型和状态结构较稳定。后端 19 项测试中有 1 项 embedding 维度测试失败，该失败被记录为后续修复项，而不是在论文中隐藏。

这些结果说明项目已经达到可演示、可本地运行和可继续迭代的状态。实践论文关注的是系统是否做成、效果如何以及问题如何被发现和改进，因此测试失败项本身也是实践证据：它指出长期记忆模块还需要更强的配置约束。

6. 项目发布与总结

6.1 发布与运行情况

当前项目以本地运行为主要交付方式。后端通过 FastAPI 启动 API 服务，前端通过 Vite 启动 React 页面，用户在本机浏览器中访问界面并与 Live2D 角色对话。项目已经具备课程演示所需的核心链路：登录、创建会话、发送消息、接收 SSE 回复、播放语音、驱动表情、调用工具和保存记忆。由于涉及大模型 API、语音服务和本地 Live2D 资源，公开部署还需要进一步处理密钥、资源路径和用户数据隐私。

6.2 项目成果总结

最成功的设计决策是把 ShinobuChat 定位为低打扰桌面伴侣，而不是无限制自动化助手。这个定位让系统在工具能力、记忆能力和主动服务之间保持边界：用户明确请求行动时才调用工具，表达情绪时优先陪伴，稳定偏好才进入长期记忆。第二个成果是多模态闭环已经跑通，文本、语音、表情和口型不再是彼此孤立的功能，而是同一回复事件链路的不同表现形式。第三个成果是项目形成了可迁移组件，如 ToolRegistry、SkillRegistry、MemoryService、SSE 事件协议和 Live2D 表情映射。

6.3 不足与展望

项目仍存在不足。第一，真实用户验证样本较少，还不足以证明长期使用中的主动服务边界；第二，表情标签集合较粗，复杂情绪和混合语气仍可能匹配不准；第三，长期记忆还缺少用户可视化管理入口；第四，公开发布需要处理 API 密钥、隐私、跨平台打包和资源加载问题。

后续可以从四个方向继续完善：一是修复 embedding 维度配置问题，并增加记忆查看、删除和关闭入口；二是扩展 emotion 标签和 Live2D 表情映射，让角色表达更细腻；三是整理安装脚本、演示视频和 README，使项目更适合作作品集展示；四是探索屏幕上上下文理解和更稳定的本地模型方案，让桌面伴侣能够在用户授权下理解当前工作场景。

参考文献

- [1] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv, 2024.
- [2] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- [3] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv, 2023.
- [4] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
- [5] Wang L, Ma C, Feng X, et al. A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 2024.
- [6] FastAPI. FastAPI Documentation. <https://fastapi.tiangolo.com/>.
- [7] React Team. React Documentation. <https://react.dev/>.
- [8] Microsoft. Edge Text-to-Speech / edge-tts project documentation. <https://github.com/rany2/edge-tts>.
- [9] FunASR Team. FunASR: A Fundamental End-to-End Speech Recognition Toolkit. <https://github.com/modelscope/FunASR>.
- [10] Live2D Inc. Cubism SDK Manual. <https://docs.live2d.com/cubism-sdk-manual/>.
- [11] PixiJS Team. PixiJS Documentation. <https://pixijs.com/>.
- [12] PostgreSQL Global Development Group. PostgreSQL Documentation. <https://www.postgresql.org/docs/>.
- [13] pgvector contributors. pgvector Documentation. <https://github.com/pgvector/pgvector>.
- [14] MDN Web Docs. Server-sent events. <https://developer.mozilla.org/>.
- [15] MDN Web Docs. Web Audio API: AnalyserNode. <https://developer.mozilla.org/>.

附录

附录 A 项目仓库、运行说明与可访问方式

项目位于本地工作目录 D:\ShinobuChat。运行时需启动 FastAPI 后端和 React/Vite 前端，并配置 DeepSeek API、数据库、TTS 和 Live2D 静态资源。课程提交时可通过本地演示或导出的运行截图证明系统可运行。

附录 B 系统截图与图示清单

本文已在正文中插入主界面、系统架构、意图识别与安全、记忆检索、多模态数据流、Live2D 效果、迭代过程 and 用户验证等图示，数量不少于 5 张。

附录 C 用户调研原始数据说明

用户验证采用三名匿名目标用户 A、B、C 的任务走查和评分整理方式。原始评分在第五章表 5.1 中汇总，反馈问题和改进措施在表 5.2 中呈现。